Quest 6: The Nullspiracy

Lord of the Wings · following ingredient records forwards and backwards

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: given "item is made from these ingredients" records, count what is reachable from `NULL` (Part I), what can reach `NULL` (Part II), and what can reach two different versions of `NULL` (Part III).
Input: one record per line, `ITEM:ING1,ING2,...`, with four-letter identifiers.
Output: a count of distinct identifiers.
What changes between parts: Part I walks *down* from `NULL` to its ingredients. Part II walks *up*: which products contain `NULL` somewhere inside. Part III splits `NULL` into three versions and asks which products contain at least two of them.

1. The story

At a roadside inn the egg starts smoking near the bread, so the company suspects contaminated food. The innkeeper has records of what every product was made from, and a suspicious identifier, `NULL`, keeps turning up. The heroes first check what `NULL` is made of, then what is made *with* it, and finally learn that there are three look-alike versions of `NULL`, and it's mixing versions that is dangerous.

2. Understanding the question

The records form a graph

Each record `BRTS:FLWR,SLTT,MLKK` is a set of arrows: `BRTS → FLWR`, `BRTS → SLTT`, `BRTS → MLKK` ("is made from"). All the records together form a **directed graph**. A few things the puzzle points out:

- An ingredient can be used by many items, so the same node can be reached along several paths. Count it **once**.
- The graph can have **cycles** (in the sample, `BRDS` uses `YSTS` and `YSTS` uses `BRDS`). A walk that doesn't remember where it has been would go round forever.
- An identifier that never appears on the left of a `:` is a basic ingredient with no arrows out of it.

"Used directly or indirectly" means "reachable by following arrows". The set of everything reachable from a node is its **transitive closure**.

Forwards vs backwards

Part I follows the arrows forwards from `NULL` (what is it made of?). Part II asks the opposite: from which items can you follow the arrows and arrive at `NULL`? That's the same as following the arrows **backwards** from `NULL`.

Extra examples of my own (all run through the real code)

Part	Records	Answer	Why it's interesting
I	`NULL:AAAA` / `AAAA:BBBB` / `BBBB:NULL`	2	The chain loops back to `NULL`. `NULL` itself is not counted, and the loop doesn't run forever.
I	`NULL:AAAA,BBBB` / `AAAA:CCCC` / `BBBB:CCCC`	3	A "diamond": `CCCC` is reached twice but counted once.
I	`AAAA:BBBB` (no record for `NULL`)	0	Nothing to expand.
II	`NULL:XXXX` / `AAAA:BBBB` / `BBBB:AAAA` / `CCCC:NULL`	1	Only `CCCC`. The `AAAA`/`BBBB` cycle never reaches `NULL`.
II	`NULL:AAAA` / `AAAA:NULL` / `BBBB:AAAA`	2	`AAAA` and `BBBB`. `NULL` is in a cycle with itself but isn't counted as a product.
III	`PIEE:NULLv1,NULLv1`	0	The same version twice is still only one version.
III	`JAMM:NULLv1` / `TART:JAMM,NULLv2` / `CAKE:TART`	2	`TART` (v2 directly, v1 through `JAMM`) and `CAKE` (both through `TART`).

Shared code: reading the records (identical in all three parts)

```ts
import { readFileSync } from "node:fs";

const isSample = process.argv.includes("--sample");
const file = isSample ? "sample1.txt" : "input1.txt";
const INPUTS: string[] = readFileSync(file, "utf8").trim().split("\n");

const items: {
  [key: string]: string[];
} = {};

INPUTS.forEach((i) => {
  const [item = "", ingredients = ""] = i.split(":");
  items[item] = ingredients.split(",");
});
```

q6/part1.ts, lines 1–14

Lines	What happens
3–5	`--sample` picks the sample file; otherwise the real input. The file is split into lines.
7–9	`items` maps an identifier to the list of its direct ingredients (an **adjacency list**).
11–14	Each line is split at `:` into the item and its ingredient list, which is split at `,`. So `BRDS:FLWR,SLTT,YSTS` becomes `items.BRDS = ["FLWR", "SLTT", "YSTS"]`. Basic ingredients get no entry, so `items["BTTR"]` is `undefined`.

3. Part I: what is NULL made of?

The question

Starting from `NULL`, collect every ingredient you can reach by following the records, counting each one once and leaving out `NULL` itself. How many are there?

Worked example (sample)

Level	Found by expanding	New ingredients
1	`NULL`	`BRDS`, `FLWR`, `SLTT`
2	`BRDS`, `FLWR`, `SLTT`	`YSTS`, `MLKK`, `BTTR`
3	`YSTS`, `MLKK`, `BTTR`	`SPCS` (`MLKK` also gives `NULL`, which is skipped)
4	`SPCS`	nothing: `SPCS` is a basic ingredient

Total: 3 + 3 + 1 = **7**, matching the puzzle.

Step-by-step solution

1. Read the records into a map from item to its list of direct ingredients.
2. Start with `NULL`'s list.
3. In each round, append the ingredients of every entry in the list.
4. Remove `NULL`, empty strings and duplicates.
5. If the list didn't get longer, stop. Otherwise do another round.
6. Print the length of the list.

My solution

```
16  let added = true;
17
18  while (added) {
19    added = false;
20
21    const originalLength = items["NULL"]?.length || 0;
22
23    items["NULL"]?.forEach((i) => {
24      if (i !== "NULL") items["NULL"]?.push(...(items[i] || []));
25    });
26
27    items["NULL"] = [
28      ...new Set((items["NULL"] || []).filter((i) => i !== "NULL" && i)),
29    ];
30
31    added = originalLength != (items["NULL"]?.length || 0);
32  }
33
34  console.log(items["NULL"]?.length || 0);
```

q6/part1.ts, lines 16–34

Lines	What happens
16–19	Keep doing rounds while the previous round found something new.
21	Remember how long the list is at the start of the round. `?.` and `|| 0` cover the case where there is no `NULL` record.
23–25	For every entry currently in `NULL`'s list, append that entry's own ingredients to the end of the same list. `forEach` decides how many elements to visit **before** it starts, so the entries pushed during this round are not visited until the next round. Each round therefore goes exactly one level deeper.
27–29	Clean up: drop `NULL` itself and any empty strings (`&& i`), then remove duplicates with a `Set`. A `Set` keeps the first occurrence of each value in order, so the old entries stay at the front.
31	If the cleaned list has the same length as at the start, nothing new was found, and the loop stops. This is safe because entries are never removed (apart from `NULL`/empty strings, which were removed already), so "same length" means "same set".
34	The answer is the length of the list.

Trace: rounds of the `while` loop (sample)

Round	Length at line 21	Length after pushing (23–25)	Length after cleaning (27–29)	Keep going?
1	3	11	6	yes
2	6	18	7	yes
3	7	19	7	no → prints 7

After round 1 the list is `BRDS`, `FLWR`, `SLTT`, `YSTS`, `MLKK`, `BTTR`; round 2 adds `SPCS`. In round 2, the raw list contains `NULL` (from `MLKK`), and line 28 removes it again.

> **Why the cycle doesn't hang:** `BRDS` → `YSTS` → `BRDS` is a cycle, but each round just re-adds things that are already in the list, the `Set` throws away the duplicates, and the length stops growing. The loop ends when a whole round finds nothing new: a **fixed point**.

> **Cost:** each round expands *every* entry in the list again, not just the ones found in the last round. That's repeated work: a round costs about as much as the whole list's records, and there's one round per level. The usual way to avoid it is a **breadth-first search** with a "seen" set and a queue, where each item is expanded exactly once. For an input this size, your version is still instant.

4. Part II: what uses NULL?

The question

Now go the other way: count the products (other than `NULL`) that contain `NULL` directly or indirectly.

Worked example (sample)

Turning the records around ("who uses X?") and walking up from `NULL`:

Level	Used by	New products
1	`NULL` is used by	`MLKK`
2	`MLKK` is used by	`SLTT`
3	`SLTT` is used by	`BRDS`, `FLWR`, `HNYN` (and `NULL`, not counted)
4	`BRDS`, `FLWR` are used by	`YSTS`

Total: 1 + 1 + 3 + 1 = **6**, matching the puzzle. Your solution gets the same answer a different way: instead of reversing the arrows, it computes every product's forward closure and checks whether `NULL` is in it.

Step-by-step solution

1. Read the records as in Part I.
2. Turn Part I's loop into a function: given a starting list, return everything reachable from it (keep `NULL` in the result).
3. For every product except `NULL`, compute the full closure of a **copy** of its list, so the stored records stay unchanged.
4. If `NULL` is in it, count the product.
5. Print the count.

My solution

Lines 1–14 are the shared code (with `sample2.txt` / `input2.txt` on line 4).

```
16  const getFullIngredients = (ingredients: string[]) => {
17    let added = true;
18
19    while (added) {
20      added = false;
21
22      const originalLength = ingredients?.length || 0;
23
24      ingredients?.forEach((i) => {
25        if (i !== "NULL") ingredients?.push(...(items[i] || []));
26      });
27
28      ingredients = [...new Set((ingredients || []).filter((i) => i))];
29
30      added = originalLength != (ingredients?.length || 0);
31    }
32
33    return ingredients;
34  };
35
36  let noOfNullProduct = 0;
37
38  Object.entries(items).forEach(([item, ingredients]) => {
39    if (item === "NULL") return;
40
41    if (getFullIngredients([...ingredients]).includes("NULL")) noOfNullProduct++;
42  });
43
44  console.log(noOfNullProduct);
```

q6/part2.ts, lines 16–44 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
16–34	`getFullIngredients` is Part I's loop turned into a function that works on any starting list instead of only `items["NULL"]`.
25	Expands every entry except `NULL`. Not expanding `NULL` is a small shortcut: once `NULL` is in the list, the answer for this product is already "yes".
28	Unlike Part I, this keeps `NULL` in the list (only empty strings are dropped), because line 41 needs to find it.
36–42	For every product except `NULL` itself (line 39), compute its full closure and count it if `NULL` is in there. Line 41 passes a **copy** of the record (`[...ingredients]`), so the function can push onto it without changing `items` (see the note below).

Trace: one call per product (sample)

Product	Direct ingredients	Full closure (line 41)	Has NULL?
BRDS	`FLWR`, `SLTT`, `YSTS`	`FLWR`, `SLTT`, `YSTS`, `MLKK`, `BTTR`, `BRDS`, `SPCS`, `NULL`	yes
FLWR	`SLTT`, `YSTS`	`SLTT`, `YSTS`, `MLKK`, `BTTR`, `BRDS`, `FLWR`, `SPCS`, `NULL`	yes
SLTT	`YSTS`, `MLKK`, `BTTR`	`YSTS`, `MLKK`, `BTTR`, `BRDS`, `FLWR`, `SPCS`, `NULL`, `SLTT`	yes
NULL		skipped (line 39)	
YSTS	`BRDS`, `FLWR`	`BRDS`, `FLWR`, `SLTT`, `YSTS`, `MLKK`, `BTTR`, `SPCS`, `NULL`	yes
MLKK	`SPCS`, `NULL`	`SPCS`, `NULL`	yes
HNYN	`BRDS`, `FLWR`, `SLTT`, `YSTS`	`BRDS`, `FLWR`, `SLTT`, `YSTS`, `MLKK`, `BTTR`, `SPCS`, `NULL`	yes

Every product in the sample uses `NULL`, so the answer is 6. Note that a product can appear in its own closure (e.g. `BRDS`) because of the cycles; that's harmless here.

> **Why line 41 passes a copy (`[...ingredients]`):** in the first round, line 25 pushes onto the array it was given. Without the copy, that array would be `items[item]` itself, so every call would quietly make the stored record longer. An earlier version of this code did exactly that. On the sample, `items.BRDS` grew from 3 entries to 10, and because later products copied already-grown records, it compounded: `items.YSTS` reached 19 entries and `items.HNYN` 47. On the real input, the records grew about 7× in total.
> **That version still got the right answer:** everything pushed into a record is something that record could already reach. Adding shortcuts to things that are already reachable doesn't change any closure. But it made later calls slower, and it only worked by luck: a function that changes data it doesn't own is easy to break later.
> With the copy, an instrumented run shows that `items` is exactly the same after the loop as after parsing. (Part I still grows `items["NULL"]` in place, but that's on purpose there, and nothing else reads it.)

> **Cost:** this does a full closure for *every* product, so it's roughly (number of products) × (size of a closure). The reverse-arrows idea from the worked example does one search from `NULL` over a **reversed graph** instead, touching each record once. On the real input, your version runs in about a second.

5. Part III: two different versions of NULL

The question

`NULL` now comes in three versions: `NULLv1`, `NULLv2` and `NULLv3`. Count the products that use **at least two different versions**, directly or indirectly.

Worked example (sample)

Product	Versions reachable	Count	Counted?
MEAL	v1, v2, v3 (through `CAKE`, `DOUG`)	3	yes
BRDS	v1 (through `DOUG`)	1	no
CAKE	v2, v3 directly; v1 through `DOUG`	3	yes
DOUG	v1 directly	1	no
FLWR	v1 (through `DOUG`)	1	no
YSTS	none	0	no
SOUP	v1, v2, v3 (through `CAKE`)	3	yes

Three products qualify, matching the puzzle's **3**. In my own example `PIEE:NULLv1,NULLv1`, the count is 1, not 2: it's the number of *different* versions that matters.

Step-by-step solution

1. Read the records as in Part I.
2. Use the same closure function, but don't expand the three `NULL` versions.
3. For every product that isn't a version, compute the full closure of a copy of its list.
4. Count how many of `NULLv1`, `NULLv2`, `NULLv3` are in it.
5. If it's at least 2, count the product. Print the count.

My solution

Lines 1–14 are the shared code (with `sample3.txt` / `input3.txt` on line 4).

```ts
const NULLs = ["NULLv1", "NULLv2", "NULLv3"];

const getFullIngredients = (ingredients: string[]) => {
  let added = true;

  while (added) {
    added = false;

    const originalLength = ingredients?.length || 0;

    ingredients?.forEach((i) => {
      if (!NULLs.includes(i)) ingredients?.push(...(items[i] || []));
    });

    ingredients = [...new Set((ingredients || []).filter((i) => i))];

    added = originalLength != (ingredients?.length || 0);
  }

  return ingredients;
};

let noOfNullProduct = 0;

Object.entries(items).forEach(([item, ingredients]) => {
  if (NULLs.includes(item)) return;

  let noOfNullVersion = 0;

  getFullIngredients([...ingredients]).forEach((i) => {
    if (NULLs.includes(i)) noOfNullVersion++;
  });

  if (noOfNullVersion >= 2) noOfNullProduct++;
});

console.log(noOfNullProduct);
```

q6/part3.ts, lines 16–52 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
16	The three versions, in a list so they can be checked with `includes`.
18–36	The same closure function as Part II. Line 27 now skips expanding any of the three versions (they have no records anyway).
41	The versions themselves aren't products, so they're skipped.
43–47	Get the closure of a **copy** of the record (line 45, same reason as Part II). Count how many of the closure's entries are a `NULL` version. The closure has been through a `Set` on line 30, so each version appears at most once, and this counts **different** versions.
49	At least two different versions → count the product.

Trace: closures (sample, from an instrumented copy)

Product	Full closure	Versions (line 46)
MEAL	`BRDS`, `CAKE`, `DOUG`, `FLWR`, `NULLv2`, `NULLv3`, `YSTS`, `SLTT`, `NULLv1`, `SPCS`	3
BRDS	`DOUG`, `FLWR`, `YSTS`, `SLTT`, `NULLv1`, `BRDS`, `SPCS`	1
CAKE	`DOUG`, `FLWR`, `NULLv2`, `NULLv3`, `YSTS`, `SLTT`, `NULLv1`, `BRDS`, `SPCS`	3
DOUG	`FLWR`, `YSTS`, `SLTT`, `NULLv1`, `BRDS`, `DOUG`, `SPCS`	1
FLWR	`BRDS`, `DOUG`, `FLWR`, `YSTS`, `SLTT`, `NULLv1`, `SPCS`	1
YSTS	`SPCS`	0
SOUP	`BRDS`, `CAKE`, `DOUG`, `FLWR`, `NULLv2`, `NULLv3`, `YSTS`, `SLTT`, `NULLv1`, `SPCS`	3

> **The copy on line 45 matters here too:** as in Part II, `[...ingredients]` keeps the records unchanged. Without it, the records grew about 7× on the real input, and the run took about 5.1 seconds. With the copy, it takes about 4 seconds, and the answer is the same.

> **Another way to think about it:** do the Part II "walk backwards" search three times, once from each version, and count how many products are reached by two or more of the searches. That's three searches over the reversed graph instead of one closure per product.

6. Summary

Part	Question	Direction	How your code does it	Sample
I	ingredients of `NULL`	forwards from `NULL`	grow `NULL`'s list level by level until its length stops changing	7
II	products that use `NULL`	backwards to `NULL`	forward closure of every product (from a copy of its record); check for `NULL`	6
III	products using ≥ 2 versions	backwards to each version	forward closure of every product (from a copy of its record); count distinct versions	3

Key ideas: the records are a directed graph with cycles; "directly or indirectly" means the **transitive closure**; a `Set` (or a "seen" set) is what makes cycles and repeated paths safe; and asking "who uses X" is the same as searching forwards in the **reversed graph**.